

PitForge: Exact Ultimate-Pit Optimization Reproducing the Published MineLib Optima, with Whittle Nested Shells and Scheduling

Felipe Santibañez-Leal 

Abstract—The first optimization of open-pit mine design is the ultimate pit limit: given a block model in which each block carries a net economic value and a set of slope-precedence constraints, find the set of blocks to extract that maximises total value while respecting the slopes. The problem has an exact polynomial solution as the maximum-weight closure of the precedence graph, computed as a minimum cut, and PitForge implements it to run *live in the browser*: drag the price, revenue factor or slope and the exact pit re-solves instantly. This report validates the solver against ground truth and extends it to the two decisions that follow. The validation is the strongest kind available: on three of MineLib’s eleven instances (newman1, zuck_small and kd, those for which a verified public mirror exists, 1000 to 14000 blocks), the solver reproduces the *published* optimum to a relative error near 10^{-10} , an exact match, and solves the largest of the three in about a fifth of a second. Beyond the single pit, the tool computes the Whittle nested pit shells, the family of optimal pits over an ascending revenue-factor schedule, with their value, tonnage and strip-ratio curves that guide phase and pushback design, and it solves two explicitly separated constrained production-scheduling scenarios. On the published newman1.cpit scenario (six periods, 8% discount and two resources), it reproduces the published 24,486,184 LP bound and reports a feasible 23,553,245 heuristic, a 3.81% gap. A separate synthetic twin (eight periods, 10% discount and one resource) has an 11.29% gap and is labelled non-comparable. Every pit value is self-checked (pit value = $\sum$ positive value – max flow is asserted on every solve), MineLib is CC BY-SA 3.0 Unported; the project fetches instances into a local cache and commits only attributed summaries, and every figure regenerates from the committed artifacts.

Index Terms—Open-pit mining, ultimate pit limit, maximum closure, minimum cut, nested pit shells, Whittle, production scheduling, MineLib, reproducibility.

I. INTRODUCTION

THE ultimate pit limit (UPL) is the foundational optimization of open-pit mine design: given a three-dimensional block model in which each block has a net value (recoverable metal less mining and processing cost) and slope-precedence constraints (a block can be mined only after those above it in the pit wall), which set of blocks maximises total value? The problem has an exact, polynomial-time solution, established by Lerchs and Grossmann [1] and understood through Picard’s theorem [2] as the maximum-weight closure of the precedence graph, which is the source side of a minimum s – t cut and

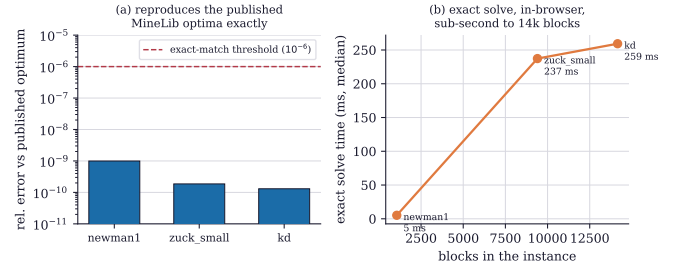


Fig. 1. Validation on the public MineLib instances. (a) The solver reproduces the *published* optimum of newman1, zuck_small and kd to a relative error near 10^{-10} , far below any exact-match threshold. (b) The exact solve is fast enough to run interactively in a browser: about 5 ms for the 1060-block newman1, and about a quarter of a second for the 14153-block kd.

hence solvable by any maximum-flow algorithm [3], [4]. This is textbook operations research [8], but three things turn the textbook solve into a useful tool: it must be validated against known answers, it must extend to the family of nested pits used for phase design, and it must connect to the production schedule.

PitForge does all three, exactly and in the browser. It solves the UPL as a minimum cut, self-checking the value identity on every solve, and it re-solves instantly as a user drags the economic parameters. This report validates it against the published MineLib benchmark optima [6], the gold standard for a pit optimiser, and reports the two extensions: the Whittle nested pit shells [5], the optimal pits across an ascending revenue factor that give the value-tonnage-strip curves phase design rests on, and a constrained production schedule [7] with its honest optimality gap. The contribution is not a new algorithm; it is an exact, validated, transparent implementation that reproduces the literature and exposes the honest gaps where they remain.

II. THE EXACT PIT, VALIDATED ON MINELIB

The ultimate pit is the maximum-weight closed set of the precedence graph, and PitForge computes it by Picard’s reduction to a minimum cut: a source arc to every positive-value block, a sink arc from every negative block, and an infinite-capacity arc for each precedence relation, after which a Dinic maximum flow [9] gives the pit as the source-reachable set, with the closure property and the value identity pit value = $\sum_{v_b > 0} v_b$ – maxflow asserted on every solve of both lanes so a wrong answer cannot be returned silently.

F. Santibañez-Leal is with the CAOS open-research programme, Santiago, Chile (e-mail: fsantibanez@gmail.com; ORCID 0000-0002-0150-3246).

Technical report, 2026. Interactive tool and reproducible pipeline (MIT) at https://github.com/fsantibanezleal/CAOS_PitForge; live at <https://pitforge.fasl-work.com>.

TABLE I

PITFORGE VERSUS THE PUBLISHED MINELIB OPTIMA. THE SOLVER MATCHES ALL THREE TO RELATIVE ERROR NEAR 10^{-10} . SOLVE TIMES ARE OMITTED FROM THIS TABLE BECAUSE THEY ARE MACHINE-DEPENDENT; THE RELEASED ARTIFACT RECORDS THEM TOGETHER WITH THE ENVIRONMENT THEY WERE MEASURED IN.

Instance	blocks	published optimum	rel. error
newman1	1060	2.6087×10^7	1.0×10^{-9}
zuck_small	9400	1.4227×10^9	1.9×10^{-10}
kd	14153	6.5220×10^8	1.3×10^{-10}

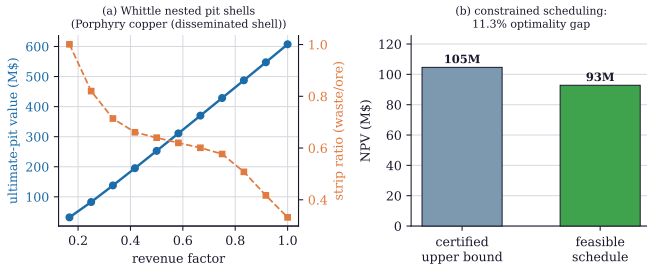


Fig. 2. Beyond the single pit. (a) The Whittle nested pit shells: solving the ultimate pit over an ascending revenue factor gives a family of nested pits whose value rises and whose strip ratio falls as the pit grows, the value-tonnage-strip curves that phase and pushback design read. (b) A constrained production schedule for the separate PitForge-authored synthetic twin over eight periods at 10% discount and one movement limit: the certified upper-bound net present value (104.6M) and the independent feasible greedy schedule (92.8M), whose 11.29% gap is reported against that scenario’s relaxation bound. It is not the published `newman1.cpit` scenario.

The validation is against the strongest available reference (Fig. 1, Table I): three of MineLib’s eleven instances, those with a verified public mirror, whose optima are published. On all three, PitForge’s value matches the published optimum to a relative error near 10^{-10} , 9.96×10^{-10} on `newman1`, 1.86×10^{-10} on `zuck_small`, 1.30×10^{-10} on `kd`, an exact match to floating-point precision. And it is fast: the 1060-block `newman1` solves in milliseconds and the 14153-block `kd` in a fraction of a second under Node, running the same TypeScript engine the browser runs, so the exact pit re-solves interactively as a user changes the economics. Those times are machine-dependent (repeat runs on one laptop varied by a factor of several), so the released artifact records its measurement environment and no decimal figure is quoted here; what is stable within a run is that the independent pseudoflow rung is one to two orders of magnitude slower on the two larger instances. Reproducing the published optima exactly is the validation a pit optimiser needs, and it is the foundation the rest of the tool builds on.

III. NESTED SHELLS AND SCHEDULING

A mine is not designed as one pit but as a sequence, and PitForge computes the two objects that sequence rests on (Fig. 2). The first is the Whittle nested pit shells [5]: solving the ultimate pit over an ascending schedule of revenue factors (multiplying the metal price) yields a family of nested optimal pits, from a small high-grade core at a low revenue factor to the full pit at a factor of one, and their value, tonnage and strip-ratio curves (Fig. 2a) are exactly what a planner uses to

choose phases and pushbacks. Because each shell is an exact ultimate-pit solve, the whole family is exact. The second is the production schedule: assigning the pit’s blocks to time periods under a capacity constraint to maximise discounted value is the constrained pit scheduling problem [7], harder than the UPL, and PitForge reports it honestly (Fig. 2b). The displayed synthetic-twin scenario has a 104.6 million bound and a 92.8 million feasible heuristic, a 11.29% gap. Separately, PitForge parses the published `newman1.cpit` definition (six periods, 8% discount, total-movement and processing limits), reproduces its 24,486,184 bound within relative error 4×10^{-9} , and finds a feasible 23,553,245 heuristic, a 3.81% gap to that bound. MineLib’s cited historical feasible value is 23,483,671 with a 4.1% gap; PitForge does not claim a new best-known result. Naming the scenario and denominator prevents the two gaps from being mixed.

IV. DISCUSSION AND SCOPE

PitForge’s value is an exact, validated, transparent pit optimiser that does the whole first stage of open-pit design in the browser and is honest about where exactness ends. The ultimate pit and the nested shells are solved *exactly*, and the MineLib reproduction to relative error 10^{-10} is the proof; the schedule is solved to a certified bound with the optimality gap reported, because the scheduling problem, unlike the UPL, does not admit a cheap exact solve at these sizes. The self-checking value identity means a returned pit value is guaranteed consistent, not merely plausible. The scope is stated plainly. MineLib is licensed CC BY-SA 3.0 Unported. As a project data-minimisation choice, source instances are fetched at runtime and only attributed summary numbers (the optima and timings) are committed. The synthetic cases used for the nested shells and the schedule are physically reasonable block models, not a specific deposit, so their numbers illustrate the method rather than predict a mine. And the tool computes the pit and the schedule; it does not replace the geological, geotechnical and financial judgement that turns them into a mine plan. None of this is hidden; the interface asserts the value identity and states the scheduling gap on every solve.

V. CONCLUSION

PitForge solves the open-pit ultimate-pit limit exactly, as a maximum-closure minimum cut, live in the browser, and reproduces the three published MineLib optima to relative error near 10^{-10} , in milliseconds to a quarter of a second. It extends the exact solve to the Whittle nested pit shells that guide phase design, and to a constrained production schedule whose gaps it reports per scenario: 3.81% for the published `newman1.cpit` scenario and 11.29% for a separate, non-comparable synthetic twin. Reproducing the benchmark optima exactly, self-checking every solve, and stating the scheduling gap honestly are the report’s discipline, and the transparent, reproducible optimiser that does it is its contribution.

APPENDIX A REPRODUCTION

The committed artifacts under `data/derived/` carry the MineLib benchmark (`minelib-results.json`: per-instance published optimum, solved value, relative error and timing), the nested-shell curves per case (`case-results.json`: revenue factor, pit value, tonnage and strip ratio), and the constrained schedule (`cpit-schedule.json`: the certified bound and rounded-schedule net present values). The figure scripts under `manuscripts/ultimate-pit/figures/` read those artifacts; the exact solver is deterministic, so re-running reproduces Table I and Figs. 1–2. The MineLib instances themselves are CC BY-SA 3.0 Unported and fetched from public mirrors into a gitignored cache by project policy.

REFERENCES

- [1] H. Lerchs and I. F. Grossmann, “Optimum design of open-pit mines,” *CIM Bulletin*, vol. 58, pp. 47–54, 1965.
- [2] J.-C. Picard, “Maximal closure of a graph and applications to combinatorial problems,” *Management Science*, vol. 22, no. 11, pp. 1268–1272, 1976. doi:10.1287/mnsc.22.11.1268.
- [3] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. doi:10.4153/CJM-1956-045-5.
- [4] D. S. Hochbaum, “The pseudoflow algorithm: A new algorithm for the maximum-flow problem,” *Operations Research*, vol. 56, no. 4, pp. 992–1009, 2008. doi:10.1287/opre.1080.0524.
- [5] J. Whittle, “Beyond optimization in open pit design,” in *Proc. Canadian Conf. on Computer Applications in the Mineral Industry*, Rotterdam: Balkema, 1988, pp. 331–337.
- [6] D. Espinoza, M. Goycoolea, E. Moreno, and A. Newman, “MineLib: A library of open pit mining problems,” *Annals of Operations Research*, vol. 206, no. 1, pp. 93–114, 2013. doi:10.1007/s10479-012-1258-3.
- [7] R. Chicoisne, D. Espinoza, M. Goycoolea, E. Moreno, and E. Rubio, “A new algorithm for the open-pit mine production scheduling problem,” *Operations Research*, vol. 60, no. 3, pp. 517–528, 2012. doi:10.1287/opre.1120.1050.
- [8] A. M. Newman, E. Rubio, R. Caro, A. Weintraub, and K. Eurek, “A review of operations research in mine planning,” *Interfaces*, vol. 40, no. 3, pp. 222–245, 2010. doi:10.1287/inte.1090.0492.
- [9] E. A. Dinic, “Algorithm for solution of a problem of maximum flow in networks with power estimation,” *Soviet Mathematics Doklady*, vol. 11, pp. 1277–1280, 1970. (Pre-DOI translation; no open landing page.)